



LECTURE 8

1) FINAL PROJECT (BIT MORE)

2) G-RETRIEVER (GRAPH LLM) WALKTHROUGH

3) KG-RAG INFERENCE: IMPLEMENTATION WALKTHROUGH

DATASCI290: NEUROSymbolic AI

FEBRUARY 2 2026



ON THE FINAL PROJECT ...



EVALUATION

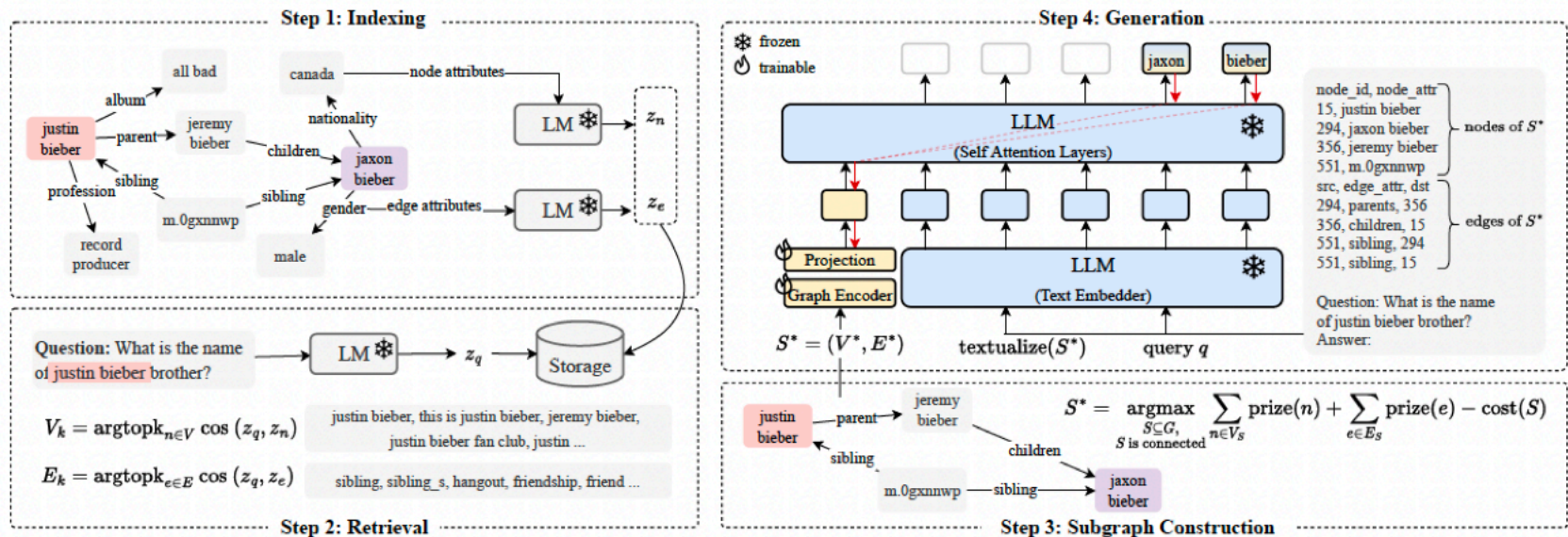
- We will evaluate CareTrace head-to-head against a strong baseline: a state-of-the-art conversational model (for example, ChatGPT or Gemini) used as-is in the same multi-turn scenario, with the same information available to the user.
- Evaluation Dimensions
 - **Confidence** (of the plan): Does the recommendation inspire appropriate confidence because it is coherent and consistent across turns and aligns with cautious clinical reasoning?
 - **Actionability**: Does it produce a concrete, stepwise plan a caregiver can execute immediately, including what to monitor and what would change the recommendation?
 - **Efficiency** (of the conversation): How quickly does it converge to the right plan with minimal back-and-forth, asking only the questions that truly matter?
 - **Focus**: Does it stay on track as the conversation evolves, or does it wander, repeat itself, or create cognitive overload at the moment the caregiver most needs clarity?
 - **Safety** (with Clarity !): How clear and unambiguous are the escalation triggers, and how well do they map to the caregiver's real decision of whether it is safe to wait until morning?



G-RETRIEVER ALGORITHM

WHAT WAS IT :) ?

RECAP: CORE ALGORITHM



■ Problem setting

- • Large attributed graph (text on nodes/edges) + natural-language QA.

■ Pipeline

- Step 1: Embed nodes + question → retrieve candidates.
- Step 2: Embed nodes + question → retrieve candidates.
- Step 3: Select connected evidence subgraph via PCST.
- Step 4: Represent subgraph for LLM (text + optional graph token) → generate answer.

LET'S TAKE A (TINY !) CARETRACE KG

- **Nodes**

- **Symptoms / signs**

- Fever
 - Vomiting
 - DecreasedUrineOutput (oliguria)

- **Risks / states**

- Dehydration
 - KidneyInjuryRisk

- **Actions**

- GiveOralRehydrationSolution
 - GiveAcetaminophen
 - AvoidIbuprofen

- **Thresholds**

- NoUrine8Hours (an escalation trigger)

- **Edges**

- DecreasedUrineOutput -> suggests -> Dehydration
 - Vomiting -> contributes_to -> Dehydration
 - Dehydration -> increases_risk -> KidneyInjuryRisk
 - KidneyInjuryRisk -> contraindicates -> Ibuprofen
 - Dehydration -> treat_with -> GiveOralRehydrationSolution
 - Fever -> symptomatic_tx -> GiveAcetaminophen
 - Dehydration -> implies -> AvoidIbuprofen
 - NoUrine8Hours -> escalation -> ER_or_urgent_eval

THE USER QUESTION (INPUT TO THE SYSTEM)

- Example question (multi-turn collapsed to one query for simplicity):
 - *“My 6-year-old has fever and vomited once; he’s only urinated once today. Can I give ibuprofen tonight?”*
- This is the “X” in the paper (question tokens)

STEP 1: EMBED – THE KG AND THE QUESTION

- What “embed nodes” means here
 - Each node gets a vector embedding (from a text encoder, or learned in training). Same dimensionality as your retriever embedding space.
 - Let retriever embedding dimension be $d_r = 768$ (typical).
 - Each node v has embedding $e(v) \in \mathbb{R}^{768}$
 - The question is embedded as $q \in \mathbb{R}^{768}$
- Conceptually:
- $q = \text{encode_text}(\text{"fever vomited once urinated once can I give ibuprofen"})$
- Then compute similarity scores:
 - $\text{score}(v) = \text{cosine}(q, e(v))$

STEP 2: RETRIEVE – RELATED PARTS OF KG

- The system picks top-K nodes by similarity.
- For this question, plausible top hits might be:
 - Ibuprofen (if present as node; or AvoidIbuprofen)
 - DecreasedUrineOutput
 - Dehydration
 - Vomiting
 - Fever
 - GiveOralRehydrationSolution
 - KidneyInjuryRisk
 - Acetaminophen
- So the output of Step 2 is something like:
 - *Candidate set $C = \{DecreasedUrineOutput, Dehydration, Vomiting, Fever, AvoidIbuprofen, GiveORS, KidneyInjuryRisk, GiveAcetaminophen\}$*

STEP 2 RETRIEVAL: TYPICALLY ...

- Often, in retrieval we “expand candidates”
 - By grabbing neighbors / multi-hop expansions around these hits
 - To ensure **connectivity**
 - The best explanation is usually a *connected* chain, not isolated nodes !
- For example:
 - If **Dehydration** is a candidate, add its “1-hop” neighbors:
 - KidneyInjuryRisk, GiveOralRehydrationSolution, AvoidIbuprofen, NoUrine8Hours
- So after expansion, you have a “candidate induced subgraph” on maybe 20–100 nodes
 - A few hundred nodes is still too big to dump into the LLM !

STEP 3: SELECT A CONNECTED EVIDENCE SUBGRAPH VIA PCST (PRIZE-COLLECTING STEINER TREE)

■ Intuition

- Each candidate node gets a “prize” (how relevant it is to the question).
- Each edge has a “cost” (penalize adding too many / irrelevant hops).
- PCST finds a connected subgraph (often tree-like) that:
 - keeps high-prize nodes
 - pays edge costs only when needed
 - produces a compact connected “evidence subgraph”

■ What it looks like numerically (micro example)

- Suppose we have prizes (higher = more relevant):
 - DecreasedUrineOutput: 0.95
 - Dehydration: 0.90
 - AvoidIbuprofen: 0.85
 - KidneyInjuryRisk: 0.70
 - Vomiting: 0.65
 - GiveOralRehydrationSolution: 0.55
 - Fever: 0.40
 - GiveAcetaminophen: 0.35
 - NoUrine8Hours: 0.30
- Edge costs might be something like 0.10–0.25 per hop.

PLAUSIBLE PCST OUTPUT

- PCST might choose the connected chain that most directly supports “avoid ibuprofen + hydrate + escalation rule”.
- **Selected subgraph S^*** (nodes + edges):
 - **Nodes:**
 - DecreasedUrineOutput
 - Dehydration
 - Vomiting
 - KidneyInjuryRisk
 - AvoidIbuprofen
 - GiveOralRehydrationSolution
 - (optionally) NoUrine8Hours (if the model decides escalation thresholds are important for this question)
 - **Edges:**
 - DecreasedUrineOutput -> suggests -> Dehydration
 - Vomiting -> contributes_to -> Dehydration
 - Dehydration -> increases_risk -> KidneyInjuryRisk
 - KidneyInjuryRisk -> contraindicates -> Ibuprofen (or to AvoidIbuprofen)
 - Dehydration -> treat_with -> GiveOralRehydrationSolution
- (optional) NoUrine8Hours -> escalation -> ER_or_urgent_eval
- Step 3 output: **a compact, connected, evidence-ready subgraph.**

STEP 4: REPRESENT THE SUBGRAPH FOR THE LLM (TEXT + OPTIONAL GRAPH TOKEN) → ANSWER

- 4A) Textualized subgraph (this part is easy !)
 - Convert S^* into a structured text block. A “CSV-like” list might look like:
 - Evidence:
 - (DecreasedUrineOutput) -[suggests]-> (Dehydration)
 - (Vomiting) -[contributes_to]-> (Dehydration)
 - (Dehydration) -[increases_risk]-> (KidneyInjuryRisk)
 - (KidneyInjuryRisk) -[contraindicates]-> (Ibuprofen)
 - (Dehydration) -[treat_with]-> (GiveOralRehydrationSolution)
 - (NoUrine8Hours) -[escalation]-> (UrgentEvaluation)
- *Then* the LLM prompt might be:
- **Prompt text:**
- System: “You are a careful pediatric triage assistant. Use the evidence to answer.”
- User:
 - Evidence: (the lines above)
 - Question: “Can I give ibuprofen tonight?”
- This is “normal RAG style” but using graph-derived evidence

4B: GRAPH TOKEN AS A “SOFT PROMPT”

- A “graph token” is **not** a word like “GRAPH_TOKEN” in the prompt.
 - It’s an **extra learned embedding vector** (numbers) that gets inserted into the *embedding sequence* that the LLM processes.
- **4B.1 Encode the subgraph into a single vector**
 - Run a **graph encoder** (e.g., a GAT) over S^* .
 - Input: node features (initial embeddings) + edges of S^*
 - Output: per-node hidden states $H \in \mathbb{R}^{|V^*| \times d_g}$ where d_g might be, say, 256.
- Then pool to a single graph representation:
 - $g = \text{pool}(H) \in \mathbb{R}^{d_g}$
 - Say $g \in \mathbb{R}^{256}$
- We can think of g as: “*a dense summary of the retrieved subgraph’s structure and semantics.*”

4B.2 PROJECT GRAPH VECTOR INTO THE LLM EMBEDDING SPACE

- LLM embeddings live in a different dimension, call it d_l
 - For many LLMs, d_l might be 4096 (varies by model).
- Learn a **projection matrix**:
 - $W \in \mathbb{R}^{d_l \times d_g}$ and bias $b \in \mathbb{R}^{d_l}$
- Then compute:
 - $t = W g + b$
 - $t \in \mathbb{R}^{d_l}$
- This t is the **graph token embedding**.
- **What does the “soft prompt vector” look like?**
- It's literally a vector like:
 - $t = [0.021, -0.113, 0.004, 0.087, \dots]$ of length d_l (e.g., 4096 floats)

4B.3 HOW IT GETS USED BY THE LLM

- In an LLM, text is turned into *embeddings* before *attention layers*.
- Let:
 - Text tokens = $x_1..x_p$
 - Their embeddings = $X_e \in \mathbb{R}^{p \times d_l}$
- The graph token is *one more embedding row*:
 - $t \in \mathbb{R}^{1 \times d_l}$
- The LLM actually receives:
 - $[t ; X_e] \in \mathbb{R}^{p \times d_l}$
- So it's like prepending one extra “virtual token” at the embedding level.
- Note: We **cannot** do this via a plain chat API
 - We do it in model code (PyTorch / HF Transformers) where we can feed `inputs_embeds` instead of `input_ids`
 - Or we modify the embedding layer to include prefix vectors.

G-RETRIEVE: SUMMARY

- **Why both text and graph token?**

- Because they carry different information:

- Text evidence: explicit facts, readable, directly citable
- Graph token: structural + relational patterns (and a learned bias toward “how to use graph evidence”)

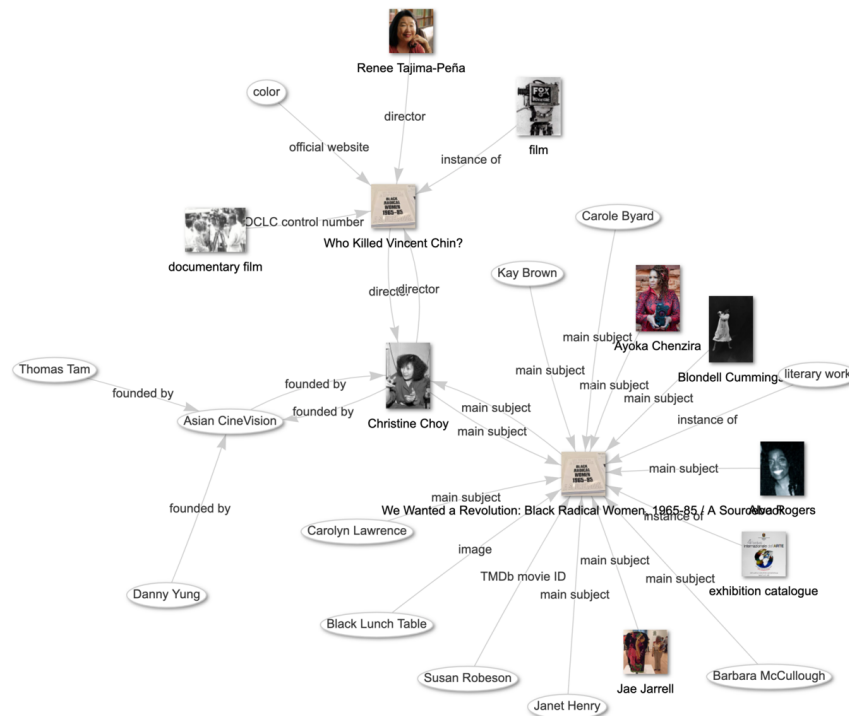
- In our CareTrace graph, the token can help the model consistently prioritize the “dehydration → kidney risk → avoid ibuprofen” chain even if the textual evidence is short or noisy.

- **Simple Recap (Steps)**

- 1. Embed nodes + question
“Put the question and each KG node into the same vector space so we can score relevance.”
- 2. Retrieve candidates
“Grab the top relevant nodes (and neighbors) to form a candidate region.”
- 3. PCST subgraph selection
“Choose a small connected subgraph that preserves the best evidence chain without dragging in the whole KG.”
- 4. Feed subgraph to LLM
“Give the evidence as text, and optionally also as a learned ‘graph token’ embedding prepended to the LLM’s embedding sequence.”

A GOOD LARGE KG: IN CASE YOU WANT TO TRY THIS OUT :)

- Wikidata Knowledge Graph
- <https://dumps.wikimedia.org/wikidatawiki/entities/>





KG-RAG

LET'S TRY IT OUT



KG-BASED INFERENCE IN CARETRACE

- This walkthrough explains how CareTrace uses a knowledge graph during triage.
- We focus on three inference styles: retrieval, reasoning, and controlled generation.
- Each style trades off flexibility, safety, and explainability.

THE KNOWLEDGE GRAPH



- Nodes represent clinical concepts: symptoms, signs, conditions, actions, red flags.
- Edges encode medically meaningful relations (e.g., INDICATES, RED_FLAG_FOR, RECOMMEND).

LLM USE: OLLAMA

■ <https://ollama.com/>



METHOD 1: KG-AUGMENTED RETRIEVAL

- Goal: answer a question using directly relevant graph facts.
- We identify keywords in the user input and retrieve matching nodes and edges.
- The retrieved subgraph is converted to text and passed to the LLM as evidence.

RETRIEVAL EXAMPLE

- User input mentions fever, vomiting, low urine.
- Graph retrieval pulls dehydration, viral illness, and hydration actions.
- LLM is instructed to answer using only this evidence block.

WHAT RETRIEVAL GIVES US

- Fast, simple grounding of the response.
- Clear traceability: every recommendation maps to a graph edge.
- Limit: no multi-step reasoning beyond what is explicitly connected.

METHOD 2: KG-AUGMENTED REASONING

- Goal: reason across multiple turns and refine hypotheses.
- The agent iteratively queries the graph as new symptoms are revealed.
- Each iteration updates the working clinical picture.

REASONING IN THIS EXAMPLE

- Step 1: fever + vomiting → viral illness hypothesis.
- Step 2: low urine → dehydration risk added.
- Step 3: breathing trouble or cyanosis → immediate escalation.

(KG AUGMENTED) REASONING HAPPENS VIA A PERSISTENT STATE

- **Core idea:** We do not treat each turn as a fresh prompt; we maintain a structured **state** that accumulates facts, missing information, evidence, and the evolving plan, and we update it step-by-step until it stabilizes.
- **State = a structured snapshot** (carried *across iterations*)
 - **Patient and context facts** (normalized)
Example: age=6, fever=true, vomit_count=1, urine_output=low, time_of_day=23:00
 - **Open questions** (missing slots)
Example: hours_since_last_urine, can_keep_fluids, temp_value, breathing_distress, abd_pain_location
 - **Working concerns** (prioritized problem list)
Example: dehydration_risk=high, appendicitis_watch=medium, viral_illness_likely=medium
 - **Retrieval targets (what to query next and why)**
Example: “retrieve KG facts about dehydration and NSAID avoidance” or “retrieve escalation thresholds for low urine”
 - **Evidence trace (explicit KG facts already used)**
Example: edges supporting “avoid ibuprofen when dehydrated” and “ER threshold if no urine $\geq$ 8 hours”
 - **Plan draft (current recommendation)**
Example: hydration micro-sips, acetaminophen dosing, monitor, escalation triggers
 - **Step history (what changed each step)**
Example: “Step 1 added dehydration risk; Step 2 added NSAID contraindication; Step 3 added escalation threshold”

REASONING VIA STATE

- Each iteration = State $\rightarrow$ Retrieve $\rightarrow$ Update $\rightarrow$ State'
 - Use current state to decide **one high-value next action** (ask a question OR retrieve evidence OR refine plan)
 - Query the KG only for the **targeted evidence** needed
 - Update the state with new facts/evidence and revise the plan
- Generalizable to any number of steps
 - We repeat until a **stop condition** holds: plan stops changing, no critical missing slots remain, evidence covers all critical claims, or we hit a step budget.
- The agent becomes **adaptive** (it changes what it does next based on what it already knows) while staying **coherent** (the plan is the consequence of the evolving state, not a one-shot generation).

METHOD 3: KG-CONTROLLED GENERATION

- Goal: tightly control what the LLM is allowed to say.
- We issue a Cypher query to retrieve only allowed actions and thresholds.
- The LLM generates language, but content is constrained by the graph.

KG-RAG NOTEBOOK

- Available in Berkeley DataHub (JupyterHub)
- Folder: **KG-RAG Inference**
- Encouraged to try it out, extend it, investigate